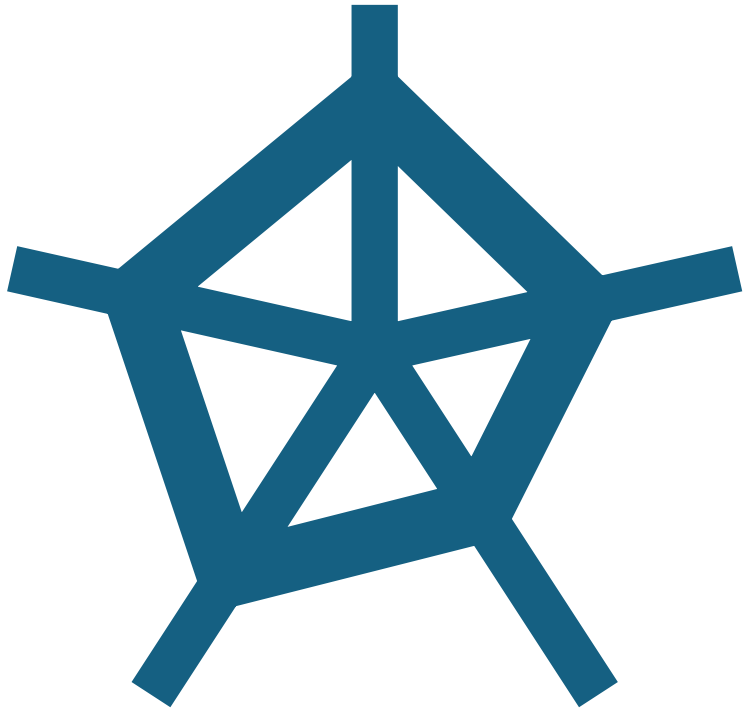


Course 11

NoSQL – Graph

Why

Traditional relational models struggle with interconnected datasets.

Graph DBs provide native mechanisms to express, store, and query complex relationships with high performance and semantic clarity.

A graph database is a data management system that stores information using nodes, relationships, and properties.

It models data as a connected network rather than rigid tables.

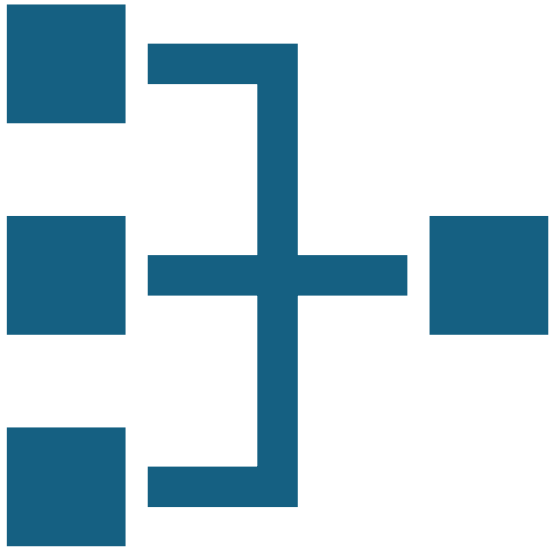
Foundation

Graph databases derive from graph theory, where data is represented as:

V = set of vertices (nodes)

E = set of edges (relationships)

This foundation enables algorithmic operations such as shortest paths, centrality, and traversal-based queries.



Components

- **Nodes** represent entities or objects.
  Examples: Person, Product, City, Device.
  Nodes can contain arbitrary key–value properties.

- **Relationships** represent directed or undirected connections between nodes.
  They can also store properties such as weights, timestamps, or categories.

- **Properties** define attributes for both nodes and relationships, providing flexibility in schema design and allowing highly expressive queries.



LPG

The property graph is the most widely used model in contemporary graph database systems such as Neo4j, TigerGraph, or JanusGraph.

It is characterized by:

- Nodes and edges as first-class entities, each capable of carrying an arbitrary set of key–value properties.
- Labeled nodes, enabling semantic grouping (e.g., Person, Paper, City).
- Typed or labeled edges, representing named relationships (e.g., FRIENDS_WITH, AUTHORED, LOCATED_IN).
- Multi-relational capabilities, meaning multiple edges of various types can exist between the same pair of nodes.

Modeling in an LPG system typically emphasizes descriptive richness and flexibility. As a result, it is well suited for domains requiring heterogeneous relationships, ad-hoc schema evolution, or expressive graph traversals.



RDF

The RDF model forms the foundation of the semantic web and is adopted by graph database systems such as GraphDB, Apache Jena, and Blazegraph.

Its structure is based on:

- Triples of the form (subject, predicate, object), which constitute a directed, labeled graph.
- Uniform representation of nodes and edges, where predicates (edge labels) and nodes are all URIs or literals.
- Schema and semantic constraints expressed through standards such as RDFS and OWL, enabling reasoning and inference.

RDF modeling emphasizes semantic interoperability, machine-understandable vocabularies, and logical inference. It is particularly effective in contexts where shared ontologies or semantic consistency across distributed datasets are required.



Hypergraph

While less common in mainstream graph database engines, hypergraph-based approaches represent relationships involving more than two entities.

In a hypergraph:

- A hyperedge can connect any number of nodes, enabling direct representation of higher-arity relationships.
- Hypernode models sometimes encapsulate sub-graphs as nodes, supporting hierarchical and modular structures.

These approaches are conceptually attractive for domains such as biological networks or knowledge representation, where natural relationships are not strictly binary.



Relation Centric

Some modeling approaches treat relationships as rich entities with their own ontological roles. While LPGs allow edge properties, relation-centric models go further by:

- Allowing edges to participate in additional relationships.
- Providing explicit identity for edges independent of node pairs.
- Supporting reification, where relationships are modeled as nodes to encode meta-relationships.

This approach is valuable for capturing temporal, provenance, or contextual information about relationships themselves; e.g., “Alice knows Bob because of Project X” or “Authored (relationship) occurred at time t”.

Schema (less)



Schema-First (Schema-Driven) Modeling

- Defines node labels, relationship types, and property constraints before data ingestion.
- Encourages consistency, formal validation, and predictable querying patterns.
- Favored in enterprise or regulated environments.

Schema-Optional (Schema-On-Read) Modeling

- Allows data structures to evolve organically.
- Defers strict schema definitions until query or analysis time.
- Suited for exploratory analytics and rapidly changing domains.

Most **LPG** databases support schema-optional modeling, whereas **RDF** environments tend to encourage schema-first modeling through formal ontologies.

Approach



Some contemporary graph systems adopt a hybrid approach, integrating graph structures with relational or document-oriented storage. In such cases:

- Graph models coexist with tabular schemas.
- Certain entities are best stored as documents or relational tables, while relationships and connectivity are represented in graph form.
- The modeling approach becomes an exercise in partitioning data across representational paradigms.

This approach is particularly common in large-scale analytics platforms and multi-model databases such as ArangoDB or Cosmos DB.

Graph vs Relational

Unlike traditional relational databases, which depend heavily on **JOIN** operations to reconstruct relationships across multiple tables, graph databases operate by directly **traversing** explicitly **stored connections** between entities. In a relational system, the process of linking data often becomes increasingly expensive as the number of tables grows or as the depth of the relationships expands. Graph databases, by contrast, maintain pre-materialized edges that encode these relationships natively in the data model.

As a result, queries involving complex networks such as multi-hop paths, densely interconnected records, or variable-length relationship patterns can be executed with substantially higher performance. The traversal-based execution model avoids the computational overhead of repeated JOIN computations and scales more naturally with the structural complexity of the dataset. This makes graph databases particularly effective for domains where connectedness is central to the analytical workload.

Use Cases

- Social Networks

Graph databases naturally express relationships like friendships, followers, communities, and interests, enabling complex analytics.

- Fraud Detection

Financial institutions analyze relationships among accounts, transactions, and devices to detect suspicious patterns using graph traversals.

- Knowledge Graphs

Knowledge graphs integrate vast heterogeneous datasets.

Search engines, recommendation systems, and semantic services rely heavily on graph structures.

- Supply Chain and Logistics

Graph models capture supplier-manufacturer-distributor relationships, enabling route optimization and dependency analysis.

Query Languages

In the ecosystem of graph data management, two query languages have emerged as the most influential and widely adopted:

- **Cypher**, originating from the Neo4j platform,
- **Gremlin**, developed as part of the Apache TinkerPop framework.

Although both languages allow users to explore and manipulate graph-structured data, they embody distinct philosophical and operational approaches.
Cypher is a **declarative, pattern-matching language** that enables users to specify the desired structural configuration of a query such as paths, motifs, or node–relationship patterns without needing to articulate the exact steps for retrieving them.

Gremlin, in contrast, adopts a **procedural, traversal-oriented paradigm**, in which the user describes an explicit sequence of navigation operations that move from vertex to vertex through the graph.

Neo4J

Neo4j is the most widely adopted native graph database.
It offers ACID transactions, a native graph engine, and the Cypher language.

Neo4J Overview

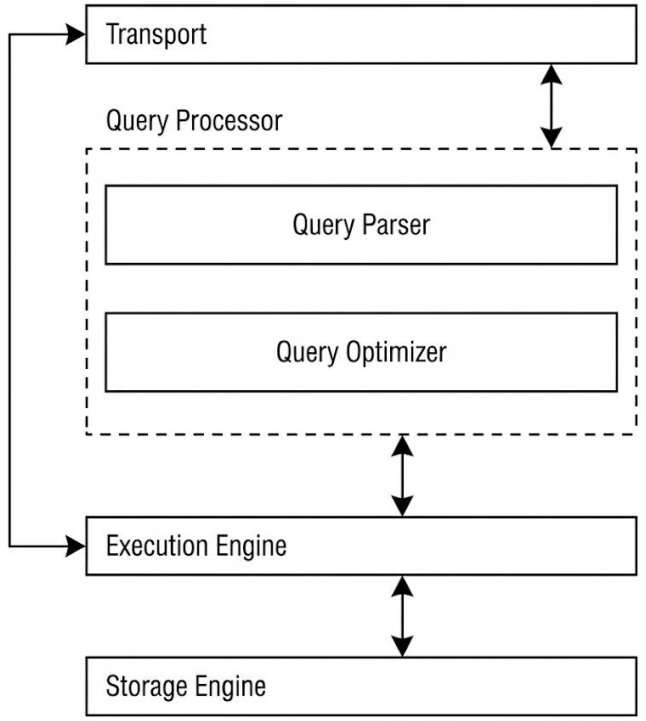
Neo4j’s architecture is built around a **native graph storage engine** and a **graph-optimized processing layer** that store and traverse nodes and relationships without translation overhead. It uses an **ACID-compliant transactional core** supported by a **clustered, highly available deployment model** for scalability and fault tolerance. Together, these components enable efficient graph traversal performance while preserving strong consistency guarantees.

Neo4J Architecture

Transport: at the beginning of any interaction with the database system, a client-issued query enters through the transport subsystem, which subsequently forwards it to the query processor for interpretation and execution.

Query Processor: the query processor parses and validates the incoming statement before the query optimizer determines an efficient execution strategy. In Neo4j, this role is fulfilled by the Cypher Query Planner, which relies on internal statistics, indexes, constraints, and data distribution to guide its optimizations.

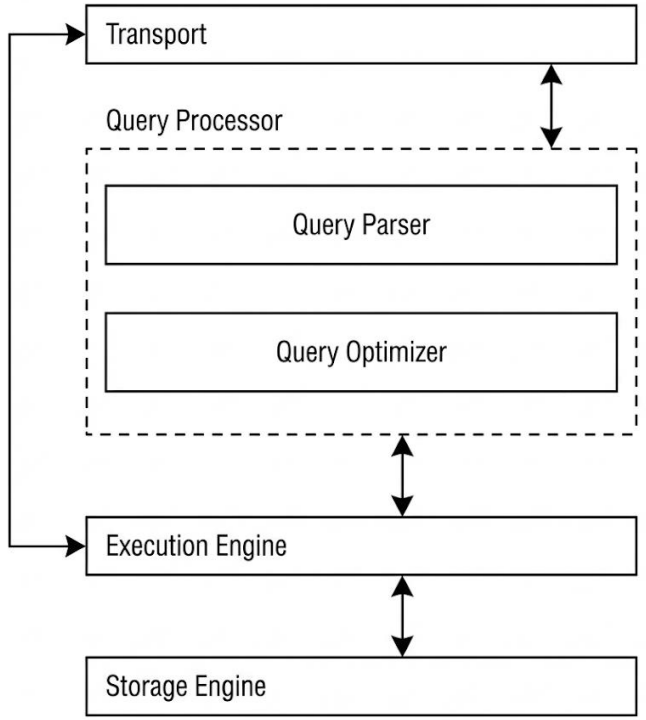
Execution Engine: the query processor produces an execution plan, a structured sequence of operations, which is then carried out by the execution engine to generate and aggregate results. This plan is composed of discrete operators arranged in a tree-like structure, where leaf operators resolve records and propagate their outputs upward, while non-leaf operators consume the intermediate results produced by their branches.



Neo4J Architecture

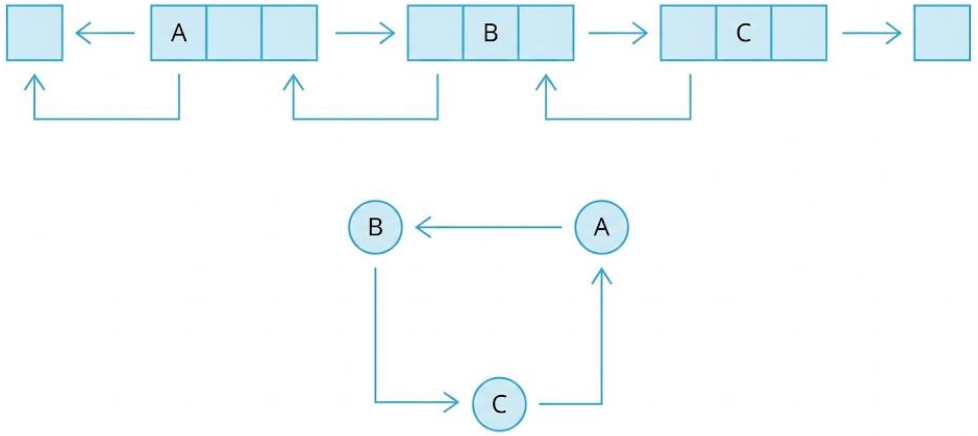
Storage: Data persisted on disk follows the index-free adjacency principle and is organized into store files, each dedicated to a specific record type and fixed-size record format. Although Neo4j employs a disk-based storage model, it depends extensively on in-memory caching to accelerate access, achieving its best performance when frequently accessed data resides in memory and thus minimizes disk I/O overhead.

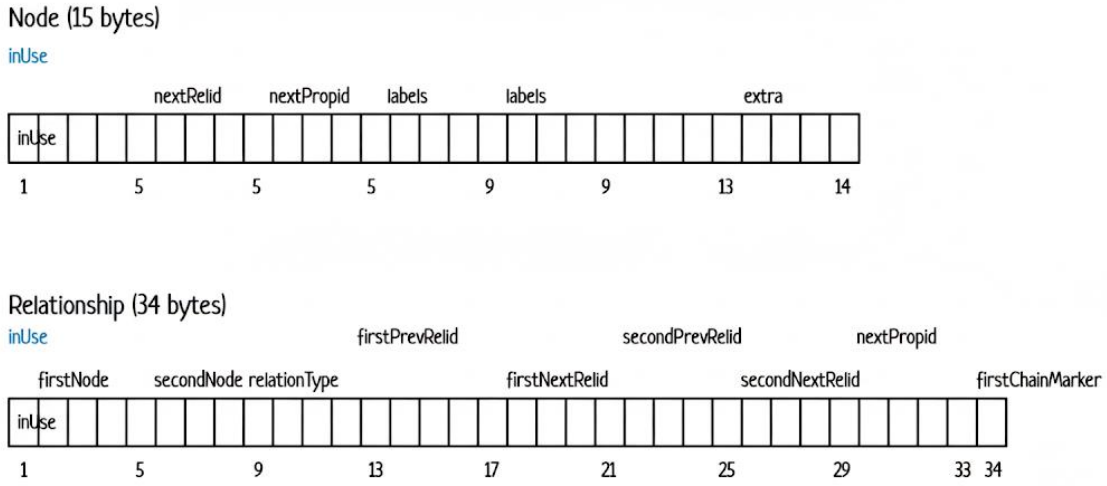
Store File	Record size	Contents
neostore.nodestore.db	15 B	Nodes
neostore.relationshipstore.db	34 B	Relationships
neostore.propertystore.db	41 B	Properties for nodes and relationships
neostore.propertystore.db.strings	128 B	Values of string properties
neostore.propertystore.db.arrays	128 B	Values of array properties
Indexed Property	1/3 * AVG(X)	Each index entry is approximately 1/3 of the average property value size



Neo4J Architecture

Indexing: is essential in traditional DBMSs for accelerating data retrieval, but large indexes can degrade write performance and consume substantial storage. Neo4j avoids these costs by relying on ***index-free adjacency***: each node directly references its neighbors through singly linked lists, while edges form doubly linked lists that link both forward and backward. This native adjacency model enables near constant-time traversal performance, as query cost depends only on the portion of the graph explored rather than on global data size.





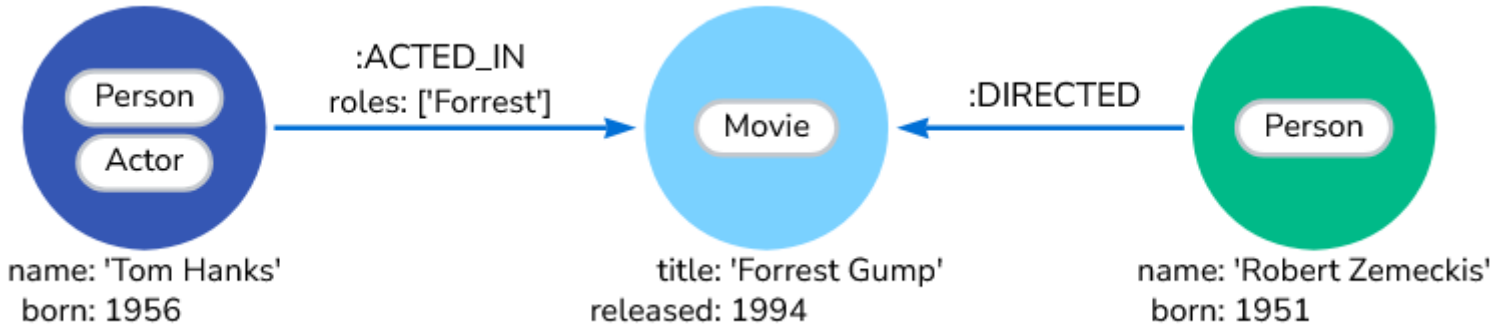
Neo4J Architecture

Neo4j's indexing relies on three global index types: b-tree, token lookup, and full-text, each optimized for different retrieval needs.

- **B-tree** indexes map property values to nodes or relationships for efficient exact-match lookups and ordered access.
- **Token lookup** indexes, created automatically for all labels and relationship types, allow rapid label-based matching and support the efficient construction of other indexes.
- **Full-text** indexes, built on Apache Lucene, serve search-oriented workloads that require flexible text matching.

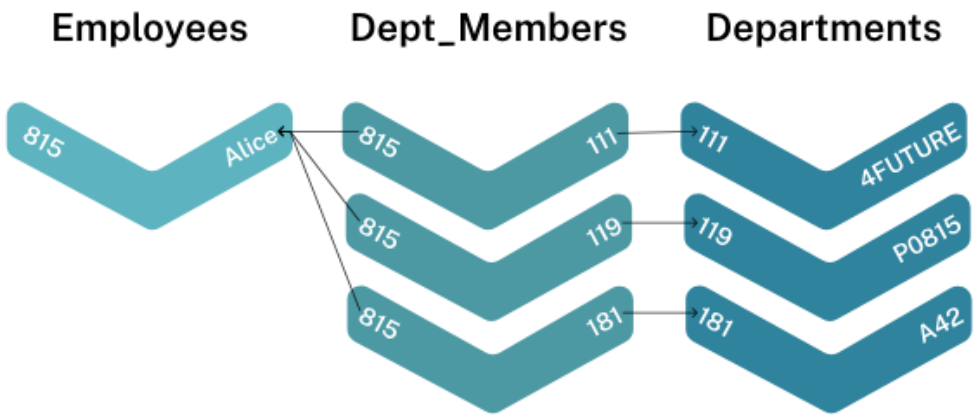
Neo4J Example Graph

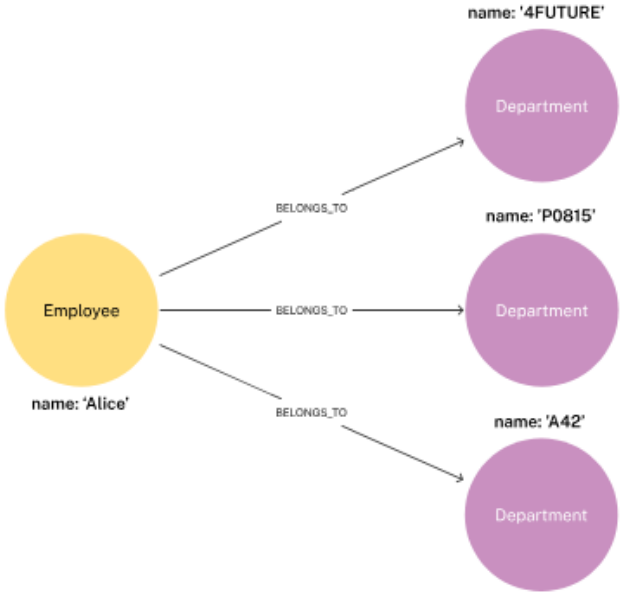
Nodes are used to represent *entities* (discrete objects) of a domain.
Node labels classify nodes into semantic groups, enabling operations to be restricted to specific sets—such as retrieving all nodes labeled *User*. Because labels can be added or removed at runtime, they also serve as markers for transient states, for example indicating suspended accounts or seasonal items. Each node may carry any number of labels, including none.
Relationships express a directed connection between a source and a target node, and may even link a node to itself. Each relationship has a single type that classifies its role and may include properties that further describe its meaning. Through these typed, property-bearing links, nodes form larger structures, ranging from lists and trees to maps and composite entities, supporting complex and densely interconnected graph topologies.
A relationship must have exactly one **relationship type**.
Properties are key–value pairs used to store descriptive information on both nodes and relationships.



Relational vs Graph

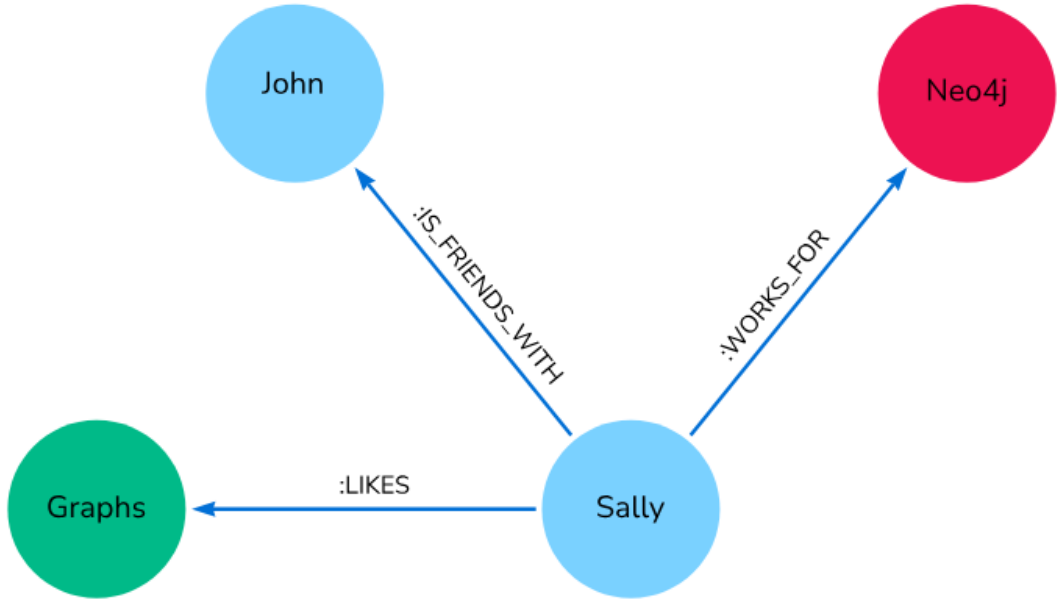
Relational databases organize data into predefined tables, requiring rigid schemas. Relationships are expressed through primary–foreign key references, with JOIN operations computed at query time by matching keys across tables, an approach that becomes computationally expensive as data complexity increases. Many-to-many relationships further amplify this cost by necessitating additional associative tables to mediate the connections. In **graph databases**, relationships are as fundamental as the data itself, eliminating the need to infer connections via foreign keys or external processing. By organizing nodes and relationships into connected structures, the database can model the problem domain directly.





Neo4J Cypher

Cypher is Neo4j's declarative query language, conforming to the GQL standard and available as open source through the openCypher project. While conceptually similar to SQL, it is specifically optimized for graph data. Its intuitive, nearly natural-language syntax uses ASCII-art–style notation to represent graph patterns and relationships, for example, (:Node)-[:RELATES_TO]->(:OtherNode), where parentheses denote nodes and square brackets with arrows indicate directed relationships. This syntax supports all standard CRUD operations on graph structures.




```
(:Sally)-[:LIKES]->(:Graphs)
(:Sally)-[:IS_FRIENDS_WITH]->(:John)
(:Sally)-[:WORKS_FOR]->(:Neo4j)
```

Neo4J Cypher

Node variables allow parts of a query to reference matched nodes in later clauses. Variables, written in lowercase, can be single letters (e.g., p:Person) or full words (person:Person). Relationships are always directional, indicated by arrows, which may point left-to-right, right-to-left, or remain unspecified for undirected links.
An undirected relationship can be traversed in either direction, even though all relationships have an inherent direction; Cypher allows querying them undirected using clauses like MATCH. This is useful when the relationship direction is unknown, ensuring all connected nodes of a given type are retrieved. Relationship types classify connections and convey meaning, typically using verbs to describe how nodes relate, making queries resemble natural language, with nodes as nouns and relationships as predicates.
Relationships can also be assigned variables, just like nodes. Once defined, these variables allow the relationship to be referenced in subsequent parts of the query.

```
MATCH (p:Person)
RETURN p
```

```
(p:Person)-[:LIKES]->(t:Technology)
```

```
(p:Person)<-[:LIKES]-(t:Technology)
```

```
MATCH (p:Person)-[:LIKES]-(t:Technology)
```

```
MATCH (p:Person)-[r:LIKES]->(t:Technology)
RETURN p,r,t
```

Neo4J Cypher

- Create nodes and relationships:

```
CREATE (p:Person {name: 'Alice’})
CREATE (p)-[:KNOWS]->
(:Person {name: 'Bob'})
```

- Matching Patterns

```
MATCH (p:Person)-[:KNOWS]-
>(f:Person)
RETURN p.name, f.name
```

- Variables

```
MATCH (p:Person {name: 'Alice'})
SET p.age = 30
```

- Querying relationship directions

```
MATCH (a)-[:KNOWS]-(b)
RETURN a, b
```

- Filtering and conditions

```
MATCH (p:Person)
WHERE p.age > 25
RETURN p
```

- CUD

```
CREATE (:Person {name: 'Alice', age:
30})
MATCH (p:Person {name: 'Alice'})
SET p.age = 31
MATCH (p:Person {name: 'Bob'})
DELETE p
```

- Return and aggregate results

```
MATCH (p:Person)-[:KNOWS]->(f)
RETURN p.name, count(f) AS num_friends
```

Cloud Graph Database Providers

Neo4j / Neo4j Aura: fully managed cloud native edition of Neo4j, available on major cloud platforms (AWS, Azure, GCP).
It preserves Neo4j's native graph storage and its expressive query language Cypher, while removing the burden of infrastructure management — which is ideal if you want scalability without operational overhead.
Amazon Neptune (on AWS): fully managed graph database supporting both property graph and RDF models. It supports multiple popular query languages (e.g. Apache TinkerPop Gremlin, SPARQL, and openCypher) enabling flexibility depending on your data model and application needs.
As a managed service it handles scaling, backups, failover, and integrates with other AWS infrastructure (security, networking, etc.), which simplifies operations for large-scale or production workloads.
Microsoft Azure Cosmos DB (Gremlin API): a multi model globally distributed database that offers graph capabilities via the Gremlin API among its supported models. That makes it a good fit if you already are invested in Azure or need a database that supports graph + other data models alongside.
Open source/self-managed but cloud deployable: JanusGraph; not strictly a "cloud service," but because it supports backends such as Cassandra, HBase or Bigtable, you can deploy it in cloud infrastructure (e.g. on virtual machines or managed clusters) and treat it as a cloud hosted graph solution.

Choices

Choose Neo4j Aura if needed native graph storage, the expressive power of Cypher, and is preferred a fully managed, cloud native graph solution without managing infrastructure.
Use Amazon Neptune when already within the AWS ecosystem, or when need support for both property graph and RDF models, or expect to scale heavily with reliability and integration with AWS services.
Go with Azure Cosmos DB (Gremlin API) when need a multimodel database in Azure and plan to mix graph data with other data paradigms (document, key-value etc.).
Opt for JanusGraph (self-managed on cloud) when need flexibility, want open-source freedom, or need to integrate graph storage with other bespoke distributed back ends or big data infrastructures.

Sample App

http://my-neo4j-movies-app.herokuapp.com/

Resources

Neo4j Getting Started
Cypher
Neo4J AuraDB